

RELATÓRIO DE DESENVOLVIMENTO: Web Scraping - Versão Avançada (API)

ALUNO: Willian Nunes dos Santos

DISCIPLINA: Business Intelligence

1. INTRODUÇÃO E OBJETIVO

O objetivo desta versão avançada foi superar as limitações do scraping via HTML extraindo diretamente de uma API "secreta" que entrega os dados em formato JSON. A aplicação coleta de forma automatizada o catálogo completo, lidando com paginação. **Website Alvo:** API de catálogo da Max Titanium (https://www.maxtitanium.com.br/api/catalog_system/pub/products/search).

Justificativa: Extrair dados diretamente de APIs JSON é consideravelmente mais rápido, estável e imune a quebras causadas por mudanças no layout visual das páginas (HTML).

2. ANÁLISE DA ESTRUTURA DOS DADOS (JSON)

Ao invés de tags HTML, os dados foram mapeados por meio de dicionários aninhados a partir da resposta JSON:

- **Nome e Link:** Acessados diretamente na raiz do item via chaves `productName` e `link`.
- **Preço e Estoque:** Encapsulados dentro de listas e dicionários internos, demandando a navegação na hierarquia: `items` -> `sellers` -> `commercialOffer` -> `Price` e `AvailableQuantity`.

3. FERRAMENTAS E BIBLIOTECAS UTILIZADAS

- **Python 3:** Linguagem base.
- **Requests:** Para simular as requisições à API, incorporando *Headers* (`User-Agent`) para simular o comportamento de um navegador real.
- **Pandas:** Para limpar duplicatas e gerar o relatório final formatado.
- **Time:** Para definir pausas (`sleep`) e não sobrecarregar ou ser bloqueado pelo servidor alvo.

4. ALGORITMO DE EXTRAÇÃO

A lógica implementada foi: A) Definição da URL da API e injeção de *headers* de navegador falso. B) Criação de um laço de repetição (*while* *continuar*) gerenciando as variáveis dinâmicas de paginação (*_from* e *_to*) de 50 em 50 produtos. C) Verificação do HTTP status code (200) e conversão direta para JSON. D) Tratamento de exceções (*try/except*) na leitura das chaves do dicionário para evitar que a falta de um atributo em um produto interrompesse o script todo. E) Lógica condicional de estoque, marcando como "Disponível" se a quantidade > 0. F) Tratamento de Strings: Conversão de ponto para vírgula no preço para aderência ao padrão contábil nacional. G) Avanço no controle de páginas e pausa de 1 segundo (*time.sleep(1)*).

5. RESULTADOS E DESAFIOS

O script efetuou com sucesso o download integral do catálogo de produtos.

- **Formato de saída:** *catalogo_maxtitanium_final.csv* contendo Nome, Preço, Status e Link.
- **Desafios enfrentados:**
 - **Paginação dinâmica:** Controlar a lógica de quebra (*break*) percebendo quando a lista retornada pela API se tornava vazia.
 - **Limpeza de dados:** Produtos com nomes duplicados, mitigado pelo uso de *df_completo.drop_duplicates(subset=['Nome'])*.
 - **Formatação robusta:** O uso combinado da troca de ponto por vírgula (*replace('.', ',')*) e o parâmetro *sep=';*' da biblioteca *pandas* garantiu a integridade perfeita da leitura no MS Excel.

6. CÓDIGO FONTE

Python

```
import requests
```

```
import pandas as pd
```

```
import time
```

```
# --- CONFIGURAÇÕES ---
```

URL da API "secreta" que entrega os dados limpos

```
base_url =  
'https://www.maxtitanium.com.br/api/catalog_system/pub/products/search'
```

Headers para simular um navegador real

```
headers = {  
    'User-Agent': 'Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36  
(KHTML, like Gecko) Chrome/120.0.0.0 Safari/537.36',  
    'Accept': 'application/json'  
}
```

Listas para controle

```
todos_produtos = []
```

```
inicio = 0
```

```
passo = 49 # A API entrega de 50 em 50
```

```
continuar = True
```

```
print("🚀 Iniciando a extração completa do catálogo...")
```

--- LOOP DE EXTRAÇÃO (PÁGINA POR PÁGINA) ---

```
while continuar:
```

```
    fim = inicio + passo
```

```
    url_paginada = f"{base_url}?_from={inicio}&_to={fim}"
```

```
    print(f"Baixando produtos {inicio} a {fim}...")
```

```
    try:
```

```
        response = requests.get(url_paginada, headers=headers)
```

```
if response.status_code == 200:

    lista = response.json()

    # Se a lista vier vazia, acabaram os produtos

    if not lista:

        print("🚩 Fim da lista de produtos.")

        continuar = False

        break

    # Processa cada produto

    for item in lista:

        try:

            nome = item.get('productName')

            link = item.get('link')

            # Lógica para pegar preço e estoque

            items = item.get('items', [])

            preco = 0

            estoque = "Indisponível"

            if items:

                sku = items[0]

                sellers = sku.get('sellers', [])

                if sellers:

                    oferta = sellers[0].get('commercialOffer', {})

                    preco = oferta.get('Price')

                    qtd = oferta.get('AvailableQuantity', 0)

                    estoque = "Disponível" if qtd > 0 else "Esgotado"
```

```
# Adiciona na lista final

todos_produtos.append({

    'Nome': nome,

    'Preço': str(preco).replace('.', ','), # Ajuste para Excel Brasil (virgula
decimal)

    'Status': estoque,

    'Link': link

})

except:

    continue


# Prepara para a próxima página

inicio += 50

time.sleep(1) # Pausa rápida para não travar


else:

    print(f"Erro na requisição: {response.status_code}")

    continuar = False


except Exception as e:

    print(f"Erro: {e}")

    continuar = False


# --- SALVANDO O ARQUIVO CORRIGIDO ---

if todos_produtos:

    df_completo = pd.DataFrame(todos_produtos)
```

```
# Remove duplicatas

df_completo = df_completo.drop_duplicates(subset=['Nome'])

print(f"\n✅ SUCESSO TOTAL! {len(df_completo)} produtos extraídos.")
display(df_completo.head())

# O PULO DO GATO: sep=';' para o Excel abrir as colunas certinhas
nome_arquivo = 'catalogo_maxtitanium_final.csv'
df_completo.to_csv(nome_arquivo, index=False, sep=';', encoding='utf-8-sig')

print(f"\n📁 Arquivo salvo como: {nome_arquivo}")
```

7. OBSERVAÇÕES

Este projeto reflete um grau elevado de maturidade na extração de dados, substituindo a raspagem HTML frágil por uma comunicação limpa e estruturada via requisições de API, incluindo boas práticas de *rate-limiting* e tratamento de inconsistências.